

## CSCI 432/532, Spring 2026

### Homework 9

Due Tuesday, March 31 at 9:00am

---

#### Submission Requirements

- Type or clearly hand-write your solutions into a PDF format so that they are legible and professional. Submit your PDF on Canvas or Gradescope.
- Put each **sub**problem on a separate page and select the corresponding problem via the Gradescope interface.
- Do not submit your first draft. Type or clearly re-write your solutions for your final submission. If your submission is not legible, we will ask you to resubmit.

#### Collaboration and Resources

The *best* resources for completing the homework are (in no particular order):

- Your own notes from lectures and problem sessions, and/or lecture recordings.
- The lecture notes linked from the course website for the lecture day.
- The questions channel on Discord.
- Office hours.
- Discussions with classmates.

You may also access almost any resource in preparing your solution to the homework. The exception is that you may not seek answers to the problems on the homework directly, such as by pasting them into a GenAI tool, searching for solutions on a search engine, looking for them on Chegg or similar websites, or asking another person for the solution.

Additionally, you **must**

- Write your solutions in your own words—this means that you should never be *copying* anything of substance from any source, and
- credit every resource you use, including GenAI tools, by providing links or full chat transcripts. If you use the provided LaTeX template, you can use the `Sources` environment for this. Ask if you need help!

Remember, if you choose to use GenAI tools, not only must you provide a full transcript of your conversation (or a link to one), you must also use the following prompt (or some variation of it), and provide a full transcript of the conversation.

Prompt:

You are acting as a teaching assistant for an advanced undergraduate/graduate algorithms and computer science theory course.

Your role is not to provide full solutions or final answers.

Instead, you should act like we are having a conversation. Ask me a single question and let me respond. Avoid asking me many questions at once or giving me a lot of information at one time. Guide me using hints, leading questions, partial insights, and sanity checks. Help me debug my own proofs, algorithms, or reductions. Do not give a complete solution or full proofs. Be patient with me. Don't give me details so that I can move on to the next part of a problem. Make sure that I understood before moving on.

The course uses Jeff Erickson's Models of Computation lecture notes, so you should guide me to answer problems using the definitions, notation, and style from those notes.

Assume I am a serious student trying to learn, not to shortcut the assignment. I am attaching the assignment, so you should refuse to provide a complete solution to the problems listed. Of course, you can walk me through the "Solved Problems" listed at the end if I ask.

Rubric for algorithms below:

**Standard algorithm rubric.**

- A complete description always includes an analysis of the algorithm's running time, even if the problem statement does not include the word "analyze". (On the other hand, unless explicitly stated otherwise, you do not need to analyze the algorithm's space usage.)
- Watch for hand-waving, pronouns without clear antecedents (especially "this"), overconfidence red flags (like "simply" or "just"), meaningless filler words (like "go through the array and"), and the Deadly Sin "repeat this process".
- Do not regurgitate standard algorithms we've already seen, or algorithms you have seen in prerequisite courses. If you use a standard algorithm as a subroutine, you can just call that algorithm by name, but be explicit about what it takes in and what it outputs, and give the runtime in the analysis.
- Every algorithm must be clearly specified in English, unless the specification is already given precisely in the problem statement. A description of the algorithm is not enough; we want a description of what your algorithm computes, not just an explanation of how your algorithm works. In particular, if the algorithm solves a more general problem than requested, you must explicitly specify the more general problem. Similarly, if the algorithm assumes any conditions that are not explicit in the given problem statement, you must state those assumptions explicitly.
- The meaning of every variable must be either clear from context (like  $n$  for input size, or  $i$  and  $j$  for loop indices) or specified explicitly.

1. Watch the following video on convolutions (and how to compute them using FFT, though we learned that in class): <https://www.youtube.com/watch?v=KuXjwB4LzSA>.

*[Hint: Use this video to understand what convolutions are and how they can be used. Then, in part (b), how can use convolutions to get information you need? (And what is the runtime of computing a convolution using FFT?)]*

2. Let  $A$  and  $B$  be two sets of  $n$  integers.
  - (a) Give a  $O(n^2 \log n)$ -time algorithm to count the distinct number of elements in the set  $A + B = \{a + b : a \in A, b \in B\}$ .
  - (b) Give a  $O(M \log M)$ -time algorithm to count the distinct number of elements in the set  $A + B$ , where  $M$  is the largest possible value of  $a + b$  for  $a \in A$  and  $b \in B$ .